REVISION SESSIONS - By Abhishriya

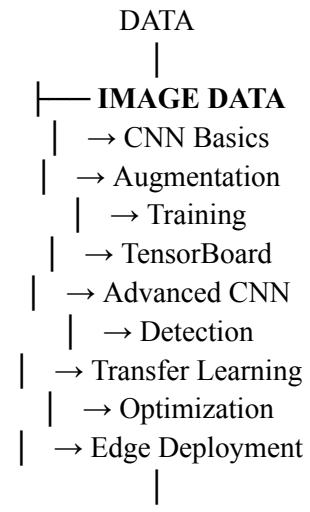
CNN Fundamentals	Convolution, kernels, stride, pooling, cats-vs-dogs CNN	Image Augmentation Lab	Albumentations, mixup, CutMix, dataset loaders	TensorBoard Vision	Filters, activations, embeddings, PR curves
Adv. CNN & Detection	Batch norm, ResNet, fine-tune, SSD, YOLO, segmentation	Transfer Learning Primer	Pre-trained weights, freezing vs finetune, LR finder	Model Optimisation	Quantisation, pruning, TensorRT, edge deploy
Sequence Models & Sentiment	RNN, LSTM, GRU, Word2Vec/GloVe, IMDb LSTM	Text Pipeline Engineering	torchtext, token buckets, batching, padding tricks	FastAPI Deployment	Inference endpoint, async queue, caching
Transformers & BERT Tune	Self-attention, positional enc., mini-transformer, BERT finetune	PEFT Hands-On	LoRA, QLoRA, adapters, PEFT library	Experiment Tracking at Scale	wandb sweeps, dataset versions, hyper dashboards
LLM Internals & Scaling	Depth/width, heads, scaling laws, interpretability probes on GPT-2	Interpreting Transformers	Attention roll-out, probing layers, causal tracing	Cost & Performance Trade-offs	Token vs context length, GPU hours, batching
Fine-Tuning LLMs & AutoML	LoRA/QLoRA, Optuna, FAQ model tune	AutoML for LLMs	Bayesian opt, early stop, search space design		

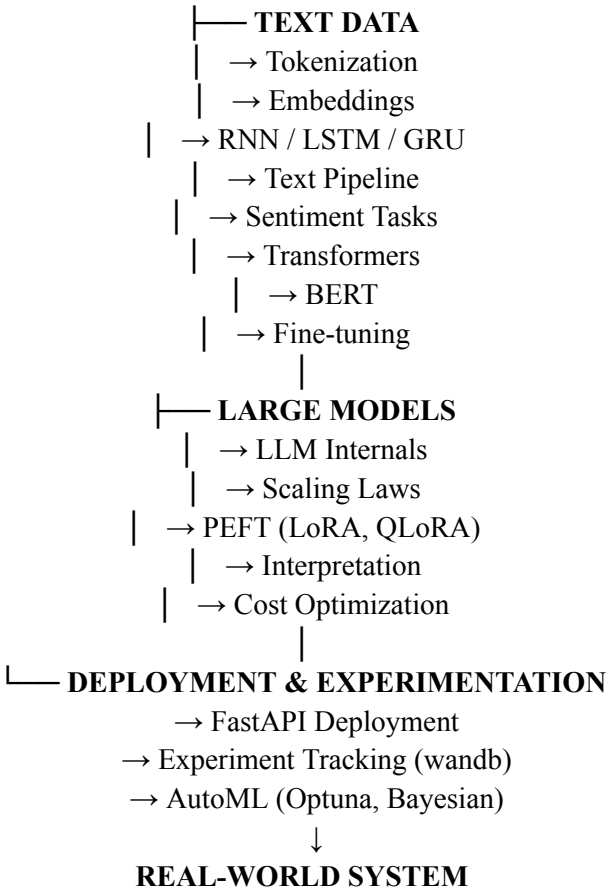
How TO STUDY

YRevision of topics and basics related questions

1. Immediatly after sessions, subjective qs of masai assignment

How Each Topics are connected for Trimester exam



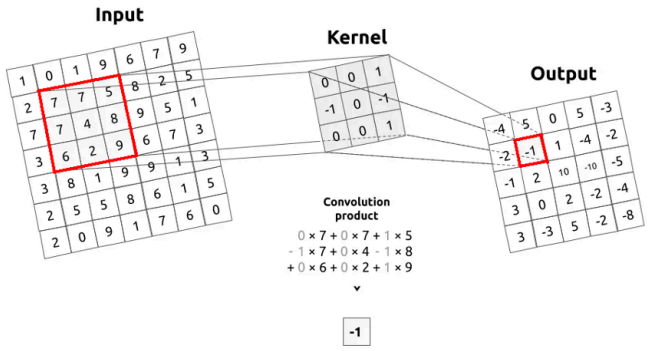


PART 1:

CNN Fundamentals (25th Feb), Adv. CNN (9th March, 11th March)

https://developers.arcgis.com/python/latest/guide/how-ssd-works/

CNN FUNDAMENTALS



1. Convolution

Idea

Small filter slides over image → extracts features

What it learns

- edges
- textures
- shapes

Example

Image of cat

- filter detects **whiskers edges**
- another detects **eyes shape**

2. Kernel (Filter)

Idea

A small matrix (like 3×3) used to scan image

How it works

Multiply + sum with image pixels

Example

Kernel for edge detection:

- highlights boundaries of dog ears or cat face

3. Stride

Idea

Step size of filter movement

- stride = 1 → slow, detailed
- stride = 2 → faster, less detail

Example

Cat image

- stride 1 → captures fine fur details
- stride 2 → skips some pixels → faster but less precise

---

4. Padding

Idea

Add zeros around image borders

Why

- keep output size same
- preserve edge information

Example

Without padding → cat ears at border may be lost
With padding → edges also learned

---

5. Activation Function (ReLU)

Idea

Add non-linearity

ReLU = remove negative values

Example

After convolution

- only strong features (like eyes, nose) remain
- weak signals removed

---

6. Pooling

Idea

Reduce size of feature map

Types:

- Max pooling
- Average pooling

Why

- reduce computation
- keep important info

Example

Max pooling keeps strongest feature

- cat eye region remains
- background noise removed

---

7. Fully Connected Layer (FC)

Idea

Final decision-making layer

Example

Features → classifier

- whiskers + ears → cat
- snout + tail → dog

---

8. Complete CNN Flow

Pipeline

Image → Conv → ReLU → Pool → Conv → FC → Output

COMMON RUNNING EXAMPLE (Cats vs Dogs)

1. Input image
2. Convolution → detects edges
3. Deeper layers → detect shapes
4. Pooling → reduces size
5. FC layer → decides label

Final output:

- Cat (0.95)
- Dog (0.05)

SIMPLE INTUITION

- Early layers → basic features (edges)
- Middle layers → shapes (eyes, nose)
- Deep layers → full object (cat/dog)

MINI FLOW (FOR STUDENTS)

Image
→ Convolution (feature extraction)
→ ReLU (important features only)
→ Pooling (reduce size)
→ Repeat layers
→ Fully Connected
→ Output (Cat / Dog)

PART 1: NUMERICAL QUESTIONS

Q1. Convolution Output Size

Input image = 32×32
Kernel = 3×3
Stride = 1
Padding = 0

Question

Find output size.

Answer

Formula:
Output = (N − F + 2P)/S + 1

= (32 − 3 + 0)/1 + 1
= 30

Final output = 30 × 30

Concept

- **No padding → size reduces**
- **Small kernel → slight reduction**

Q2. Convolution with Padding

Input = 28×28
Kernel = 5×5
Stride = 1
Padding = 2

Answer

= (28 − 5 + 2×2)/1 + 1
= (28 − 5 + 4) + 1
= 28

Final output = 28 × 28

Concept

- **Padding helps preserve size**

Q3. Stride Effect

Input = 32×32
Kernel = 3×3
Stride = 2
Padding = 0

Answer

= (32 − 3)/2 + 1
= 29/2 + 1
= 14.5 + 1 → 15 (floor)

Final output = 15 × 15

Concept

- **Larger stride → smaller output**
- **Faster but less detail**

Q4. Pooling Output

Input = 16×16
Pooling = 2×2
Stride = 2

Answer

= (16 − 2)/2 + 1
= 14/2 + 1
= 7 + 1 = 8

Final output = 8 × 8

Concept

- **Pooling halves size (commonly)**

Q5. Parameters in Conv Layer

Input channels = 3 (RGB)
Filters = 10
Kernel size = 3×3

Question

Find total parameters.

Answer

Each filter = 3×3×3 = 27 weights

- **1 bias = Total : 28**

Total = 28 × 10 = 280 parameters # weights * filters no. = No. of paarmeters.

Concept

- **Parameters depend on depth (channels)**

PART 2: CONCEPTUAL + APPLICATION QUESTIONS

Q6. Why ReLU is used?

Answer

- **Removes negative values**
- **Keeps important signals**
- **Helps model learn non-linear patterns**

Example

Without ReLU → model becomes linear
With ReLU → can detect complex shapes

Q7. Difference: Stride vs Pooling

Answer

Stride	Pooling
Used in convolution	Separate layer
Skips pixels	Reduces size
Learns features	No learning

Concept

- **Stride = movement**
- **Pooling = compression**

Q8. Why CNN instead of ANN for images?

Answer

- **CNN keeps spatial structure**
- **ANN flattens → loses position**

Example

Cat ear location matters → CNN preserves it

Q9. What problem does Augmentation solve?

Answer

- **Overfitting**
- **Limited data**

Example

1 cat image → rotated → flipped → many variations

Q10. What is Transfer Learning?

Answer

Using pre-trained model and adapting it

Example

ResNet trained on ImageNet
→ use for medical images

Concept

- **Saves time**
- **Needs less data**

PART 3: MEDIUM LEVEL THINKING QUESTIONS

Q11. If padding is not used, what happens?

Answer

- **Image size reduces**
- **Edge information lost**

Example

Cat ear near border disappears

Q12. Why deeper CNN layers learn better features?

Answer

- **Early → edges**
- **Middle → shapes**
- **Deep → objects**

Example

Edge → circle → face → cat

Q13. Why too much pooling is bad?

Answer

- **Too much information loss**

Example

Face becomes too small → model confused

Q14. What happens if stride is too large?

Answer

- **Important features skipped**

Example

Model misses cat eyes

Q15. Why batch normalization is used?

Answer

- **Stabilizes training**
- **Faster convergence**

PART 4: MIXED APPLICATION QUESTION

Q16. Design a CNN (Basic)

Question

You have 64×64 RGB image. Design simple CNN.

Answer

Step 1: Conv (3×3, stride 1) → 62×62
Step 2: ReLU
Step 3: Pool (2×2) → 31×31
Step 4: Conv → 29×29
Step 5: Pool → 14×14
Step 6: Flatten
Step 7: FC → Output

Concept

- **Gradual feature extraction**
- **Size reduction**

QUICK REVISION

- **Convolution = feature extraction**
- **Kernel = small filter**
- **Stride = step size**
- **Padding = size control**

- **Pooling = reduce size**
- **CNN = best for images**

Here are clean, compact teaching notes (less spacing, more content density) for your uploaded lecture:

ADVANCED CNN & DETECTION

---

1. Batch Normalization (BN)

Problem: Internal covariate shift → input distribution keeps changing during training → slow, unstable learning

Idea: Normalize activations → mean = 0, std = 1 (per batch), then scale + shift: to make training faster and more stable

Steps:

- Compute mean (μ) and variance (σ^2)
- Normalize: $x \rightarrow (x-\mu)/\sqrt{(\sigma^2+\varepsilon)}$
- Scale + shift: $\gamma x + \beta$ (learnable)

Example:
Values: [100,120,80,100] → normalized → [0,1.41,−1.41,0] → stable learning

Why useful:

- Faster training
- Higher learning rate possible
- Reduces dependence on initialization

Simple intuition:
Like fixing exam difficulty every week → model doesn't get confused

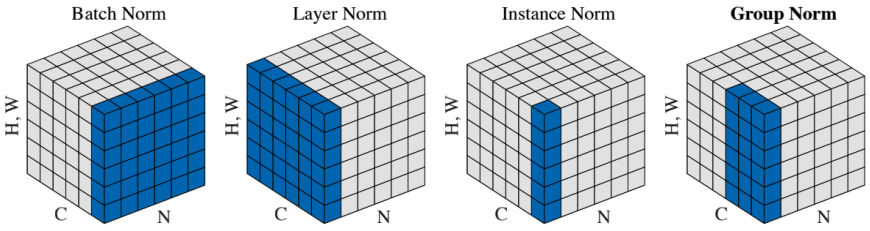


Figure 2. **Normalization methods**. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

---

2. ResNet (Residual Networks)

Problem: Deep networks perform worse (degradation, not overfitting)

Idea: Learn residual instead of full mapping

Formula:
Output = F(x) + x

Concept:

- x → skip connection
- F(x) → learned transformation

Example:
If ideal output = input → just learn F(x)=0 → easier

Why works:

- Gradient flows directly (no vanishing)
- Easier optimization

Intuition:
Instead of learning full path, learn “difference”

---

3. Transfer Learning

Idea: Use pre-trained model (ImageNet) and adapt

Why works:

- Early layers learn general features (edges, textures)
- Only final layers need task-specific learning

Strategies:

1. Freeze all → train last layer
2. Fine-tune some layers
3. Train full model

Example:
Dog breed classifier

- Already knows “ears, fur” → only learn breed differences

Benefit:

- Faster (50 hours → 50 minutes)
- Better accuracy

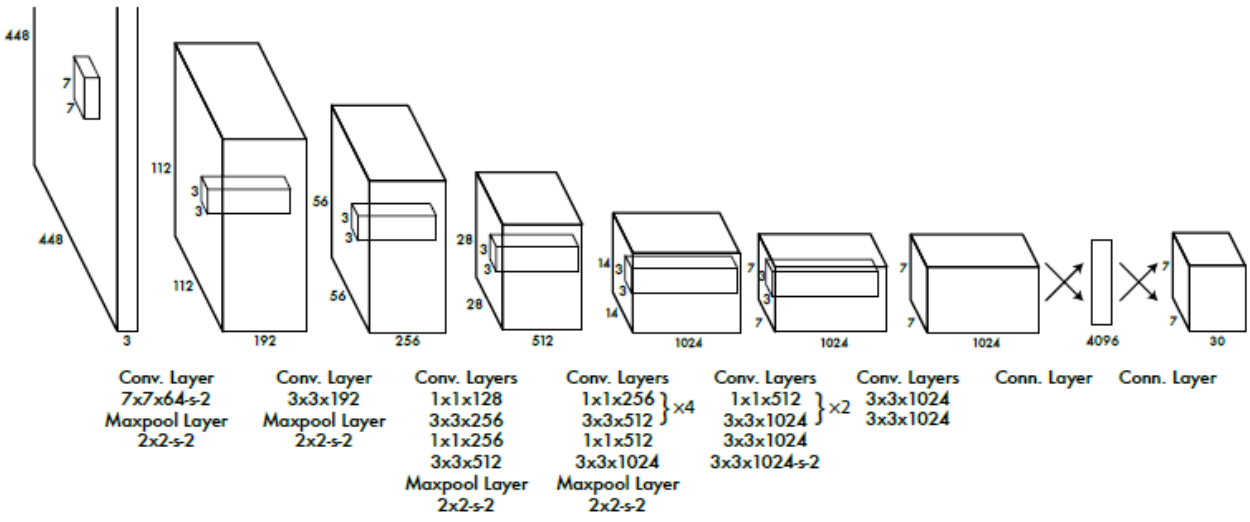
---

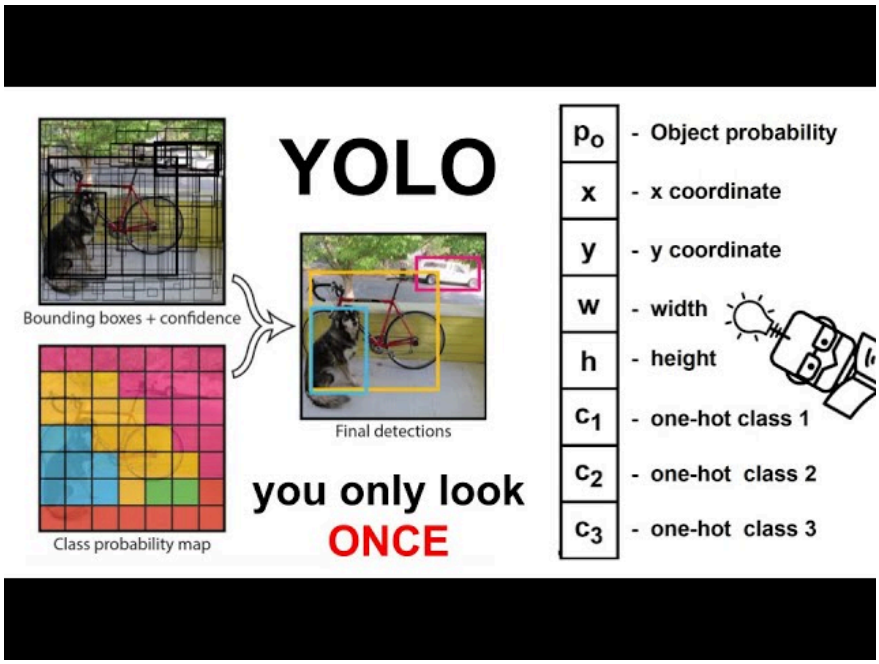
4. Object Detection

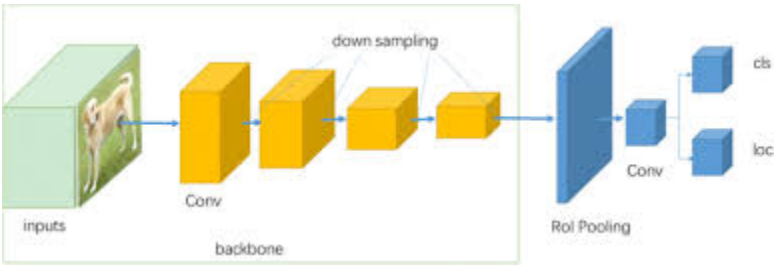
Difference from classification

- Classification → what
- Detection → what + where + how many

YOLO (One-stage) https://www.youtube.com/shorts/RVQJb1pQtw8







FIRST: WHAT IS THE TASK?

Input: Image with cars, people
Output:

- What objects?
- Where (box)?

Example:
Image → detects

- Car at (x,y,w,h)
- Person at (x,y,w,h)

PART 1: YOLO ARCHITECTURE (VERY CLEAR)

CORE IDEA (MOST IMPORTANT)

YOLO = “Look once and predict everything”

Instead of:

- Find regions → then classify

YOLO does:

- Directly predict everything in ONE STEP

STEP 1: DIVIDE IMAGE INTO GRID

Example:
Image = 448×448
Grid = 7×7

Each cell = 64×64

STEP 2: EACH CELL PREDICTS

Each grid cell predicts:

- Bounding boxes (x, y, w, h)
- Confidence (is object present?)
- Class (car, person, etc)

RUNNING EXAMPLE (CAR)

Car center at (250, 180)

Find grid cell:

- 250/64 ≈ 3
- 180/64 ≈ 2

→ Cell (3,2) responsible

That cell predicts:

- x = 0.6 (inside cell)
- y = 0.4
- w = 0.3
- h = 0.5
- class = car

STEP 3: NETWORK ARCHITECTURE

CNN PART (Feature extraction)

Image → Conv layers → feature map

448×448 → … → 7×7×1024

FC PART (Prediction)

Flatten → Fully connected

Output = 7×7×30

Why 30?

- 2 boxes × 5 values = 10
- 20 classes = 20
  Total = 30

STEP 4: OUTPUT UNDERSTANDING

Each cell gives:

[x, y, w, h, confidence] × 2 + class probs

Total predictions:
7×7×2 = 98 boxes

STEP 5: FINAL STEP

Apply NMS → remove duplicates

SIMPLE INTUITION

YOLO:

- Sees whole image once
- Predicts everything together

PROBLEM (VERY IMPORTANT)

Small objects

Example:
2 small birds in same cell
→ only one can be detected

SSD (Single Shot Detector) – Full Understanding

---

1. What is SSD?

SSD is an object detection model.
It finds:

- **what object** (class)
- **where it is** (bounding box)

In one single pass (no separate steps).

---

2. Overall Idea (Big Picture)

Input Image
→ CNN (feature extraction)
→ Multiple feature maps

→ Default boxes (anchors)
→ Predict class + location
→ Final boxes after filtering

---

3. Step-by-Step Architecture

Step 1: Input Image

Example: 300 × 300 × 3 image

This is resized before giving to model.

Step 2: Base CNN (Feature Extractor)

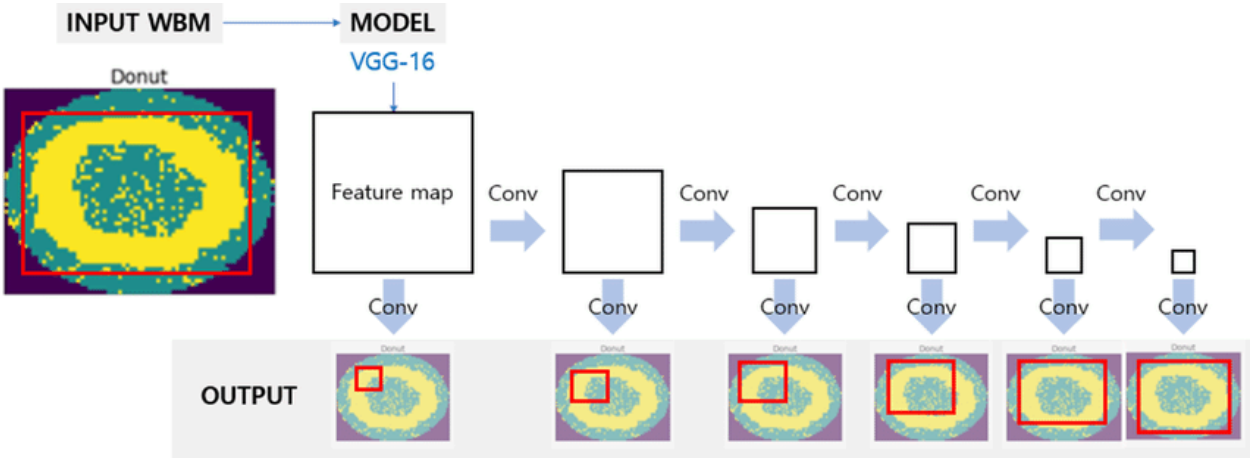
Usually uses something like VGG16

What it does:

- Extracts features (edges, shapes, objects)

Example:
Image → detects edges → textures → object parts

Output:
Feature map (smaller size, more depth)



Step 3: Multiple Feature Maps

SSD does not use only one layer.
It uses **many layers of different sizes**.

Example:

- 38×38 (detect small objects)
- 19×19
- 10×10
- 5×5
- 3×3
- 1×1

Idea:

- Large maps → small objects
- Small maps → big objects

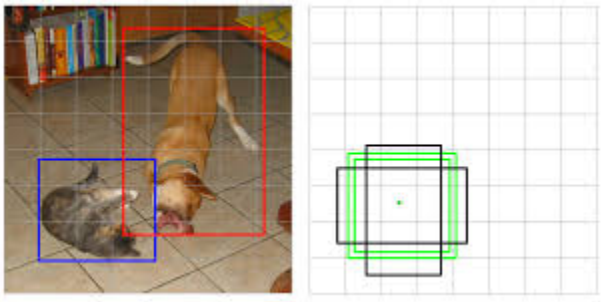
Step 4: Default Boxes (Anchors)

At each location in feature map, we create boxes.

Example:
At one point → 4–6 boxes of different shapes:

- square
- tall
- wide

These are called **default boxes**



Step 5: Predictions (Core Step)

For each default box, SSD predicts:

1. **Class scores**
   Example: dog, cat, car

2. **Bounding box offsets**
   Adjusts box position’

Idea:
Offsets = small corrections to default boxes.

Why needed?

SSD uses **default boxes (anchors)**.
But they are not perfect.

So model predicts **offsets to adjust them**.

So each box gives:

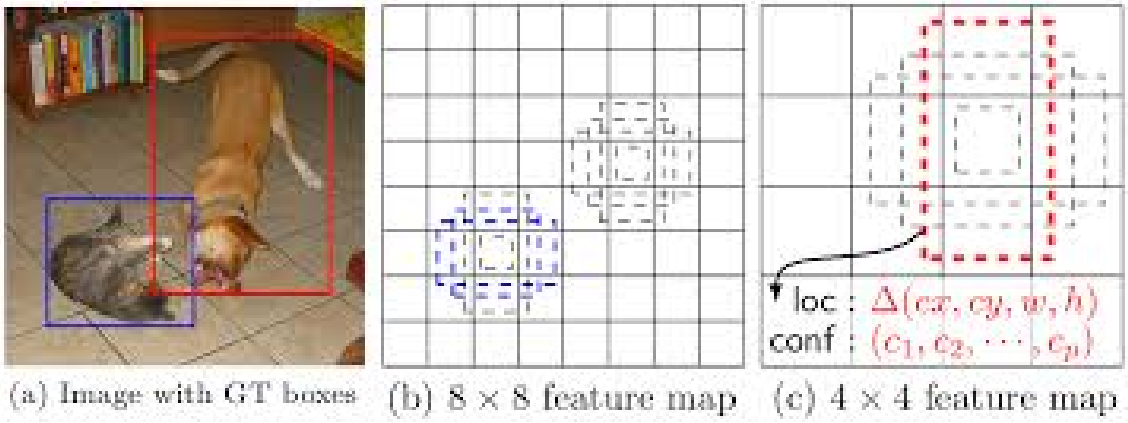
- class probability
- location correction

Step 6: Output Format

For each box:

- (cx, cy, w, h) → location
- class scores

Example:
Box → dog (0.9), cat (0.1)



(a) Image with GT boxes (b) 8×8 feature map (c) 4×4 feature map

---

Step 7: Matching with Ground Truth (Training)

During training:

Each default box is matched with real object using:

IoU (Intersection over Union)

If IoU > 0.5 → positive
Else → negative

Step 8: Loss Function

Two losses:

1. **Localization loss**
   Difference in box position
2. **Confidence loss**
   Difference in class prediction

Final loss =
Localization + Confidence

Step 9: Hard Negative Mining

Problem:
Too many negative boxes

Solution:
Keep only important negatives

Example:
Take 3 negative for 1 positive

Step 10: Non-Max Suppression (NMS)

Problem:
Many overlapping boxes

Solution:
Keep best one

Steps:

- Choose highest score box

- Remove overlapping boxes

Example:
5 boxes on same dog → keep 1

---

4. Simple Flow

Image
→ CNN
→ Feature maps
→ Default boxes
→ Predictions
→ NMS
→ Final objects

---

5. Example (Real Understanding)

Image: Dog + Cat

Step:

- Model creates 1000 boxes
- Predicts scores
- Keeps top boxes
- Removes overlap

Final:

- Dog (box)
- Cat (box)

---

6. Key Advantages

- Fast (real-time)
- Single shot (no region proposal)
- Multi-scale detection

---

7. Common Exam Questions You Can Now Solve

You can answer:

- Why multiple feature maps?
- What are default boxes?
- How SSD handles small objects?
- Difference between SSD and YOLO
- Role of IoU, NMS, loss

FINAL SIMPLE IDEA

Many boxes → predict → filter → final objects

YOLO vs SSD

Feature	YOLO	SSD
Approach	Grid	Multi-scale
Speed	Very fast	Fast
Small objects	Poor	Better
Complexity	Simple	Complex

FINAL STORY

YOLO

Teacher looks at class once
→ says: “3 students, 1 teacher”

Fast but may miss small details

SSD

Teacher:

- Looks from far
- Comes closer
- Zooms in

→ detects everyone correctly

FINAL FLOW (VERY IMPORTANT)

Image

→ CNN (features)

→ YOLO → grid prediction (fast)

OR

→ SSD → multi-scale prediction (accurate)

→ NMS

→ Final boxes

LAST SIMPLE LINE

- YOLO = fast, simple, one-shot
- SSD = multi-scale, more accurate

5. Detection Metrics

1. IoU (Intersection over Union)

Idea

Measures how much predicted box overlaps with actual (ground truth)

Formula

IoU = Intersection / Union

Example

Actual box (car): area = 100
Predicted box: area = 120
Overlap area = 80

IoU = 80 / (100 + 120 − 80)
= 80 / 140
= **0.57**

Interpretation

- IoU = 1 → perfect match
- IoU > 0.5 → correct detection
- IoU = 0 → wrong
-

Precision & Recall

- Precision = correct predictions / total predictions
- Recall = correct detections / actual objects

---

mAP (Main metric)

Measures overall detection performance across all classes

Steps (simple)

1. For each class (car, person):
    - Calculate precision & recall
    - Draw curve
    - Compute area → AP
2. Average all AP values → mAP

Example

AP (car) = 0.8
AP (person) = 0.7
AP (dog) = 0.6

mAP = (0.8 + 0.7 + 0.6) / 3
= **0.7**

Meaning

- High mAP → model detects well
- Low mAP → poor detection

---

6. Non-Maximum Suppression (NMS)

PModel predicts many boxes for same object

Example:
Car detected 4 times

- Box1: 0.95
- Box2: 0.92
- Box3: 0.88
- Box4: 0.85

Solution (Steps)

1. Pick highest confidence (Box1)
2. Remove boxes overlapping with it
3. Repeat

Example

Before: 4 boxes on same car
After NMS: 1 clean box

7. Semantic Segmentation

Idea: Classify each pixel

Example

Road image:

- Pixels of road → label = road
- Pixels of car → label = car
- Pixels of sky → label = sky

Output

Image-sized mask

FCN (Fully Convolutional Network)

Problem

Normal CNN → loses spatial info (due to flattening)

Solution

- Remove fully connected (FC) layers
- Use only convolution layers

Example

Input: 224×224 image
Output: 224×224 mask

U-Net

Structure:
Encoder (downsample) + Decoder (upsample) + Skip connections

Example:
Medical image → detect tumor boundaries

Why skip connections:

- Combine "what" + "where" → sharp output
- Input: MRI scan
  Output: tumor region
-

---

8. Key Differences (Quick Table)

Task	Output
Classification	Label
Detection	Boxes + Label
Segmentation	Pixel label

Image

→ Detection (boxes)

→ IoU (check overlap)

→ mAP (overall score)

→ NMS (remove duplicates)

→ Segmentation (pixel understanding)

→ FCN / U-Net (models)

MINI FLOW

Image

→ CNN + BatchNorm (stable learning)

→ ResNet (deep learning)

→ Transfer Learning (reuse knowledge)

→ Detection (YOLO / SSD)

→ NMS (remove duplicates)

→ Segmentation (pixel-level output)

FINAL INTUITION (VERY IMPORTANT)

- BatchNorm → stabilizes
- ResNet → enables deep learning
- Transfer learning → speeds up training
- Detection → finds objects
- Segmentation → understands every pixel

MCQs (Practice)

THEORY MCQs WITH EXPLANATION

Q1. What does IoU measure?

A. Accuracy of classification
B. Overlap between predicted and actual box
C. Speed of model
D. Number of objects detected

Answer: B

Explanation:
IoU (Intersection over Union) measures how much predicted box overlaps with ground truth.
Formula idea: overlap / total area
Used to check detection quality.

Q2. If IoU = 0.8, what does it indicate?

A. Poor detection
B. No overlap
C. Good overlap
D. Wrong class

Answer: C

Explanation:
IoU close to 1 means strong overlap.
0.8 means prediction is very accurate.

Q3. In NMS, which box is selected first?

A. Lowest confidence
B. Random box
C. Highest confidence
D. Largest size

Answer: C

Explanation:
NMS first picks the box with highest confidence score.
Then removes overlapping lower-score boxes.

Q4. Why is NMS used?

A. Increase number of boxes
B. Remove duplicate detections
C. Improve training speed
D. Reduce dataset size

Answer: B

Explanation:
Many boxes predict same object.
NMS removes duplicates and keeps best one.

Q5. mAP is calculated using:

A. IoU only
B. Loss function
C. Average Precision of all classes
D. Accuracy of one class

Answer: C

Explanation:
mAP = mean of AP values across classes.
AP comes from precision-recall curve.

Q6. Which metric uses Precision-Recall curve?

A. IoU
B. mAP
C. Loss
D. Accuracy

Answer: B

Explanation:
mAP is derived from area under precision-recall curve.

Q7. In object detection, False Positive means:

A. Correct detection
B. Missed object
C. Wrong detection
D. Perfect overlap

Answer: C

Explanation:
False positive = model predicted object but it is wrong.

Q8. Semantic segmentation outputs:

A. Single label
B. Bounding boxes
C. Pixel-wise labels
D. Only edges

Answer: C

Explanation:
Each pixel gets a class label.
Example: every pixel → road, sky, person.

Q9. FCN removes which layer?

A. Convolution
B. Pooling
C. Fully connected
D. Activation

Answer: C

Explanation:
FCN replaces fully connected layers with convolution layers to keep spatial info.

Q10. Why is FCN useful for segmentation?

A. Faster training
B. Keeps spatial information
C. Reduces data
D. Increases parameters

Answer: B

Explanation:
Segmentation needs pixel location info.
FCN preserves it.

Q11. U-Net skip connections help in:

A. Reducing computation
B. Combining low and high-level features

C. Removing noise
D. Increasing layers

Answer: B

Explanation:
Skip connections pass detailed info from encoder to decoder.

Q12. Without skip connections, output becomes:

A. Sharper
B. Blurry
C. Faster
D. Larger

Answer: B

Explanation:
Without fine details, segmentation becomes blurry.

Q13. In segmentation, which is correct?

A. One label per image
B. One label per object
C. One label per pixel
D. No labels

Answer: C

Explanation:
Segmentation = pixel-level classification.

Q14. Which task gives most detailed output?

A. Classification
B. Detection

C. Segmentation
D. Regression

Answer: C

Explanation:
Segmentation gives full pixel-level understanding.

Q15. Which statement is correct?

A. IoU measures classification accuracy
B. NMS increases duplicate boxes
C. mAP measures overall detection performance
D. FCN removes convolution layers

Answer: C

Explanation:
mAP is main metric for detection performance.

NUMERICAL MCQs WITH EXPLANATION

Q1.

Ground truth = 100, Predicted = 120, Overlap = 60
IoU = ?

Formula:
IoU = Overlap / (GT + Pred − Overlap)

= 60 / (100 + 120 − 60)
= 60 / 160
= 0.375

Closest option: **B (0.33)**

Q2.

GT = 80, Pred = 100, Overlap = 40

IoU = 40 / (80 + 100 − 40)
= 40 / 140
= 0.285

Closest: **A (0.25)**

Q3.

TP = 8, FP = 2

Precision = TP / (TP + FP)
= 8 / (8 + 2)
= 0.8

Answer: C

Q4.

TP = 8, FN = 2

Recall = TP / (TP + FN)
= 8 / (8 + 2)
= 0.8

Answer: C

Q5.

AP values: 0.6, 0.7, 0.8

mAP = (0.6 + 0.7 + 0.8) / 3
= 2.1 / 3
= 0.7

Answer: B

Q6.

Confidence: 0.9, 0.85, 0.8
All overlap high

NMS keeps highest only → 0.9

Remaining boxes = 1

Answer: A

FINAL SIMPLE IDEA

IoU → overlap quality
NMS → remove duplicates
mAP → overall performance
Segmentation → pixel-level understanding

Q7.
Image size = 448×448, grid = 7×7
Cell size = ?
A. 32
B. 64
C. 128
D. 256

Q8.
Cell (2,3), predicted x=0.5, y=0.5, grid=7
x_abs = ?
A. 0.21
B. 0.36
C. 0.50
D. 0.64

Q9.
TP = 5, FP = 5
Precision = ?
A. 0.25
B. 0.5
C. 0.75
D. 1

Q10.
TP = 5, FN = 5
Recall = ?
A. 0.25
B. 0.5
C. 0.75
D. 1

Q11.
IoU threshold = 0.5
IoU = 0.4 → prediction is?
A. TP
B. FP
C. FN
D. TN

Q12.
Total predictions = 10, correct = 7
Precision = ?
A. 0.5
B. 0.6
C. 0.7
D. 0.8

Q13.
Total GT objects = 10, detected = 6
Recall = ?
A. 0.4
B. 0.5
C. 0.6
D. 0.7

Q14.
AP (3 classes): 0.9, 0.8, 0.7
mAP = ?
A. 0.7
B. 0.75
C. 0.8
D. 0.85

Q15.
If 4 boxes overlap same object and NMS applied, expected output?
A. 4 boxes
B. 3 boxes
C. 2 boxes
D. 1 box

Tensor board tracking practically with some dataset how we visulaize the layers of CNN and track acc loss etc.

WHAT IS TENSORBOARD?

Tool to **visualize training**

- Loss
- Accuracy
- Images
- CNN filters
- Embeddings

DATASET (EXAMPLE)

Use simple dataset:
Cats vs Dogs

Folder:

```
data/
├── train/
│   ├── cat/
│   └── dog/
└── val/
```

STEP 1: BASIC TRAINING SETUP (PyTorch)

```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from torch.utils.tensorboard import SummaryWriter
```

STEP 2: LOAD DATA

```
transform = transforms.Compose([
```

```
  transforms.Resize((128,128)),
  transforms.ToTensor()
])

train_data = datasets.ImageFolder("data/train", transform=transform)
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
```

STEP 3: SIMPLE CNN MODEL

```
class CNN(nn.Module):
  def __init__(self):
    super().__init__()
    self.conv = nn.Conv2d(3,16,3)
    self.pool = nn.MaxPool2d(2,2)
    self.fc = nn.Linear(16*63*63, 2)

  def forward(self,x):
    x = self.pool(torch.relu(self.conv(x)))
    x = x.view(x.size(0), -1)
    x = self.fc(x)
    return x
```

STEP 4: INITIALIZE TENSORBOARD

```
writer = SummaryWriter("runs/cnn_experiment")
```

STEP 5: TRACK LOSS & ACCURACY

```
model = CNN()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters())

for epoch in range(5):
  running_loss = 0
  correct = 0

  for images, labels in train_loader:
    outputs = model(images)
```

```
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, preds = torch.max(outputs, 1)
        correct += (preds == labels).sum().item()

    acc = correct / len(train_data)

    # LOG TO TENSORBOARD
    writer.add_scalar("Loss/train", running_loss, epoch)
    writer.add_scalar("Accuracy/train", acc, epoch)
```

STEP 6: VISUALIZE IMAGES

```
images, _ = next(iter(train_loader))
writer.add_images("Sample Images", images)
```

STEP 7: VISUALIZE MODEL GRAPH

```
writer.add_graph(model, images)
```

STEP 8: VISUALIZE CNN FILTERS

```
weights = model.conv.weight
writer.add_images("Filters", weights)
```

STEP 9: RUN TENSORBOARD

In terminal:

```
tensorboard --logdir=runs
```

Open browser:

http://localhost:6006

WHAT YOU WILL SEE

---

1. SCALARS (MOST IMPORTANT)

- Loss decreasing
- Accuracy increasing

If:

- Loss ↑ → problem
- Accuracy flat → underfitting

---

2. IMAGES

- Input images
- Augmented images

---

3. GRAPH

- CNN structure
- Layers flow

---

4. FILTERS

- First layer filters detect edges
- Later → shapes

PRACTICAL UNDERSTANDING

During training:

You check:

- Is loss decreasing?
- Is accuracy improving?

If problem:

Issue	Meaning
Loss not decreasing	learning problem
Accuracy stuck	model weak
Overfitting	train high, val low

SIMPLE FLOW

Dataset
→ CNN Model
→ Training
→ TensorBoard logs
→ Visualize (loss, acc, filters)
→ Improve model

FINAL INTUITION

- TensorBoard = “Live monitor of model learning”
- Helps debug training
- Helps understand CNN behavior

PART 2:

Concepts Revisons+ Objective Questions +Code based and Case based questions (Masai style)

MAIN TOPICS (LMS Check from 11th March -21st March)

1. Sequence Models & Sentiment Analysis
2. Text Pipeline Engineering
3. MLOps (Artifact System)
4. PEFT (LoRA, QLoRA, Adapters)
5. Deployment System
6. Monitoring & Optimization

ONE-LINE WHAT YOU STUDY IN EACH

1. Sequence Models & Sentiment
Learn how models like RNN, LSTM, GRU understand text order and predict sentiment.

2. Text Pipeline Engineering
Learn how raw text is cleaned, tokenized, converted to numbers, and prepared for models.

3. MLOps (Artifact System)
Learn how to save, version, track datasets and models for real-world systems.

4. PEFT (LoRA, QLoRA, Adapters)
Learn how to train large models efficiently using small updates instead of full training.

5. Deployment System
Learn how to send model to real users using APIs and inference pipelines.

6. Monitoring & Optimization
Learn how to track performance, fix errors, and update models safely.

Explanation of Sequence Models & Sentiment

17 Mar, 2026 at 8:00 PM(IST)

1. TEXT PIPELINE (FROM YOUR PDF)
text pipeline is a **sequence of steps to convert raw text into usable form**

Why needed
Computers cannot understand raw text directly
Steps:
(a) Text Collection
Data is collected from:
reviews
emails
websites
This is the starting point of any NLP system
(b) Text Cleaning
Remove noise:
punctuation
symbols
extra spaces
Example:
"Great!!!" → "Great"
Purpose:
Make data clean and consistent

(c) Tokenization
Split sentence into words

Example:
"I love AI" → ["I"
"love"
,
,
"AI"]
Purpose:
Model processes tokens, not full sentences
(d) Normalization
Convert text to standard form
Includes:
Lowercase
remove stopwords
stemming / lemmatization
Purpose:
Reduce unnecessary variation
(e) Feature Representation
Convert words → numbers
Methods:

Bag of Words
TF-IDF
Embeddings (Word2Vec, GloVe)
Important concept:
Deep learning models use **embeddings to preserve meaning + order**
(f) Sequence Model (RNN / LSTM)
From your PDF (page 17–18):
Text is sequential → order matters
So models used:
RNN
LSTM
GRU
Purpose:
Understand context
Example:
"not good" ≠ "good"
(g) Output / Prediction
Final result:
sentiment
spam detectionExample:
"I loved the movie" → Positive

Text Pipeline Diagram (Revisited)

Raw Text
↓
Cleaning
↓
Tokenization
↓
Normalization
↓
Feature Extraction
↓
Machine Learning Model
↓
Prediction / Output

Our focus is to see Machine Learning Model, used text processing.

Text Pipeline in Modern AI (LLMs)

For Large Language Models the pipeline is:

Raw Text (prompt)
↓
Tokenization
↓
Token IDs
↓
Embedding
↓
Transformer Layers
↓
Logits
↓
Softmax
↓
Next Token Prediction

2. MLOPS LAYER (ARTIFACT SYSTEM)

Now we extend pipeline to real-world system

(a) Dataset Artifact

Dataset is saved with version

Example:

dataset:v1
dataset:v2

Purpose:

Exact data tracking

(b) Model Artifact

Trained model is saved

Includes:

weights
config

Purpose:

Reusable + reproducible

(c) Version Control

Every change → new version

Purpose:

No loss of history

(d) Lineage Tracking

Connects:

Dataset → Model → Experiment

Purpose:

Full traceability

(e) Deployment

Model is used in real system

Flow:

User → API → Model → Output

(f) Monitoring

Track:

accuracy
latency
drift

Purpose:

Ensure model works properly

(g) Rollback + Update

If model fails:

→ switch to old version

Purpose:

Safe production system

FINAL UNDERSTANDING
Academic view (your PDF)
Text → Processing → Model → Output
Industry view (MLOps)
Text → Model → Tracking → Deployment → Monitoring

ONE LINE SUMMARY
It comes under:
NLP Pipeline + Sequence Models + MLOps System Design

PART 3: **MAIN TOPIC**
Parameter Efficient Fine-Tuning (PEFT) in Deep Learning Systems
20 Mar, 2026 at 8:00 PM(IST)
PEFT / LoRA / QLoRA topic

FLOWCHART (VERY SIMPLE)
START
Pretrained Model
(very large model)
Problem: Too much memory
Apply PEFT
(LoRA / QLoRA / Adapter)
Train only small parameters
|
Save Adapter
|
V Combine with Base Model
|
Prediction / Use
|
v
END

SIMPLE STORY

Imagine:
You have a **very big book (model)**
You want to update it
Option 1: Rewrite full book → very hard
Option 2: Add small notes → easy
PEFT = adding small notes instead of rewriting everything.

MAIN CONCEPTS (VERY CLEAR)
1. PROBLEM: FULL FINE-TUNING
Big models (like 70B):●

Need huge memory
Very expensive
Cannot run on normal GPU
From notes:
Training can need **hundreds of GB memory**
2. SOLUTION: PEFT
Idea:
Do NOT train full model
Train only small part
Result:
less memory
faster training
almost same accuracy
3. LoRA (MOST IMPORTANT)
Simple idea:
Instead of changing full weights
Add small matrices
Formula (concept only):
Old weight + small change**Real life example:**
Teacher writes book
Student adds sticky notes
Book = original model
Sticky notes = LoRA
Why works:
Because actual changes are small
From notes:
Only **0.1% – 1% parameters updated**
Result:
Huge saving
Example:
●
Original: 16 million params
●
LoRA: 0.1 million
4. QLoRA (ADV ANCED BUT SIMPLE)
Problem in LoRA: Still model is big
Solution:
Compress model
This is called **quantization**

Simple idea:
Store numbers in smaller size
Example:
●
●
Normal: 16-bit
QLoRA: 4-bit
Real life example:
Image compression
Same idea
Special thing: NF4
Better way to compress weights
Keeps accuracy good**Result:**
Model becomes very small
From notes:
●
14GB → 3.5GB
5. ADAPTERS
Idea:
Insert small layers between model
Example:
Big pipeline
Add small module in between
Difference from LoRA:
●
●
LoRA → inside layer
Adapter → between layers
6. TRAINING PROCESS (VERY SIMPLE)
from peft import LoraConfig, get
_peft

_
model
config = LoraConfig(r=16)
model = get
_peft

_
model(base

_
model, config)
trainer.train()
7. AFTER TRAINING
You save only small part

model.save
_pretrained("adapter")

8. INFERENCE
Load base model + adapter
from peft import PeftModel
model = PeftModel.from
_pretrained(base
_
model,
"adapter")
9. MERGING (IMPORTANT)
Combine adapter into model
Result:
-
-
no extra cost
fast inference
FINAL UNDERSTANDINGOLD WAY
Train full model
→ very expensive
NEW WAY (PEFT)
Train small part
→ cheap + fast
ONE LINE SUMMARY
PEFT = Train less → Save memory → Same performance

CONCEPTS CLEARING THROUGH QUESTIONS:

OBJECTIVE (MCQs)

Medium Level

Q1. Why is tokenization important?
A. Reduces dataset size
B. Converts text into words/tokens
C. Removes noise
D. Improves GPU speed

Answer: B
Explanation: Model reads tokens, not full sentence.

Q2. Why RNN fails in long sentences?
A. Too many parameters
B. Vanishing gradient
C. Cannot read words
D. Only works on images

Answer: B
Explanation: Old info becomes weak → model forgets.

Q3. What does LSTM forget gate do?
A. Adds new info
B. Removes old info
C. Predicts output
D. Normalizes data

Answer: B
Explanation: Controls what to remove from memory.

Q4. Why embeddings (Word2Vec/GloVe) used?
A. Reduce dataset
B. Convert words to meaningful numbers
C. Increase loss
D. Remove tokens

Answer: B
Explanation: Captures meaning of words.

Q5. What is padding used for?
A. Remove words
B. Make all sequences same length
C. Increase accuracy
D. Reduce memory

Answer: B
Explanation: Needed for batching.

Hard Level

Q6. Why BiLSTM better than LSTM?
A. Faster
B. Uses less memory
C. Uses past + future context
D. Removes tokens

Answer: C
Explanation: Understands full sentence.

Q7. Why LoRA is efficient?
A. Trains full model
B. Updates small matrices
C. Removes layers
D. Uses CPU only

Answer: B
Explanation: Only small changes → less memory.

Q8. What is NF4 in QLoRA?
A. Activation function
B. Loss function
C. Quantization method
D. Optimizer

Answer: C
Explanation: Stores weights in 4-bit efficiently.

Q9. Why artifact system needed?
A. Increase accuracy
B. Track versions

C. Reduce loss
D. Tokenize text

Answer: B
Explanation: Keeps history of data + model.

Q10. What is batching advantage?
A. Slower training
B. Faster computation
C. Removes data
D. Reduces tokens

Answer: B
Explanation: Parallel processing.

SUBJECTIVE QUESTIONS

Medium Level

Q1. Explain full text pipeline with example.
Answer:
Raw text → cleaning → tokenization → normalization → embeddings → model → output
Example:
"I love AI" → ["i","love","ai"] → numbers → LSTM → Positive

Q2. Difference between RNN and LSTM.
Answer:
RNN: forgets long info
LSTM: keeps long memory using gates

Q3. Explain padding and batching.
Answer:

Padding: make same length
Batching: process many samples together

Q4. Explain Word2Vec simply.
Answer:
Converts words into vectors based on meaning
Example: king ≈ queen

Hard Level

Q5. Design sentiment system pipeline (end-to-end).
Answer:
Text → cleaning → tokens → embeddings → LSTM → output
Then: save model → deploy API → monitor

Q6. Why PEFT better than full fine-tuning?
Answer:
Less memory, faster, similar accuracy

Q7. Explain LoRA working simply.
Answer:
Instead of changing full weights → add small matrices

Q8. Explain deployment flow using FastAPI.
Answer:
User → API → model → prediction → response

Q9. Explain monitoring in ML system.
Answer:
Track accuracy, speed, errors
If bad → update or rollback

Q10. Why sequence models needed in NLP?
Answer:
Because word order matters
Example: "not good" ≠ "good"

CASE STUDY QUESTIONS (Masai Style)

Q1. Sentiment Mistake Problem

You built an RNN model.
Input: *"This movie is not good"*
Output: **Positive**

Question:
Why did this happen? How to fix?

Answer:
RNN forgets earlier words due to vanishing gradient.
It ignores "not" and focuses on "good".

Fix:
Use LSTM/GRU because they remember long context using gates.
Also use BiLSTM so model sees both directions.

Q2. Different Length Reviews Problem

Dataset:
"I love it"
"This product is extremely bad and disappointing"

Model training is slow and inconsistent.

Question:
What is the issue and solution?

Answer:
Problem is variable length sequences.
Model cannot process unequal inputs efficiently.

Solution:
Use padding to make same length.
Use batching for faster training.
Use masking to ignore padded values.

Q3. Memory Issue While Training Large Model

You try to fine-tune a big model (like 70B).
System crashes due to GPU memory.

Question:
How will you solve this?

Answer:
Full fine-tuning uses huge memory.

Use PEFT methods:

- LoRA → train small matrices
- QLoRA → compress weights (4-bit)

This reduces memory and keeps performance similar.

Q4. Real-Time API Failure

Your sentiment model is deployed using FastAPI.
Users complain response is slow (>2 sec).

Question:
What improvements will you apply?

Answer:
Use async processing to handle multiple requests.
Add caching for repeated inputs.

Use batching for inference.
Optimize model (quantization or smaller model).

Q5. Model Works in Training but Fails in Production

Accuracy in training: 92%
Accuracy in real-world: 65%

Question:
Why and what to do?

Answer:
Data distribution mismatch (data drift).
Model trained on clean data but real data is noisy.

Solution:
Monitor data continuously.
Retrain model with new real data.
Use artifact system to track versions.

Q6. Wrong Sentiment Due to Sarcasm

Input: *"Wow great service, waited 2 hours 🙃"*
Model predicts: Positive

Question:
Why and how to improve?

Answer:
Model cannot understand sarcasm.
It only sees "great" → positive.

Solution:
Use larger datasets with sarcasm examples.
Use advanced models (BiLSTM / Transformers).
Add context-aware training.

Q7. Deployment Rollback Scenario

New model deployed → performance drops.

Question:
What should system do?

Answer:
Use version control (artifacts).
Rollback to previous stable model.
Analyze issue before redeploying.

Q8. Word Meaning Problem

Model treats “bank” same in:
“I went to bank”
“I sat on river bank”

Question:
Why and solution?

Answer:
Simple embeddings don’t capture context well.

Solution:
Use contextual embeddings (like LSTM/transformer-based).
They understand meaning based on sentence.

Q9. Training Too Slow

Training takes very long time.

Question:
How to optimize?

Answer:
Use batching → parallel processing
Use GPU

Use efficient tokenization
Use smaller sequence length

Q10. Experiment Confusion

You trained 100 models but don't know which is best.

Question:
What system should be used?

Answer:
Use experiment tracking (W&B artifacts).
Track dataset, model, accuracy, parameters.
Helps compare and reproduce results.

Exam questions like—-

Problem → Reason → Fix

This is what Masai expects in answers.

CODE-BASED CASE STUDY QUESTIONS (Masai Style)

Q1. Tokenization Error

Code:

```
texts = ["I love AI", "This is bad"]
tokens = texts.split()
```

Question: What is wrong?

Answer:
`.split()` works on string, not list.

Fix:

```
tokens = [t.split() for t in texts]
```

Explanation:
Each sentence must be tokenized separately.

Q2. Padding Issue

Code:

```
from keras.preprocessing.sequence import pad_sequences
seq = [[1,2,3], [4,5]]
pad_sequences(seq)
```

Question: Why needed?

Answer:
Sequences have different lengths.

Explanation:
Padding makes all same size → needed for batching.

Q3. RNN Model Problem

Code:

```
model = nn.RNN(input_size=100, hidden_size=10)
```

Model gives poor accuracy.

Question: Why?

Answer:
RNN cannot capture long dependencies.

Fix:

```
model = nn.LSTM(input_size=100, hidden_size=10)
```

Explanation:
LSTM remembers long context.

Q4. Embedding Missing

Code:

```
model = nn.LSTM(input_size=1, hidden_size=10)
```

Question: What is wrong?

Answer:
Text is not converted into embeddings.

Fix:

```
embedding = nn.Embedding(vocab_size, 100)
```

Explanation:
Words must be converted to vectors.

Q5. Wrong Loss Function

Code:

```
loss = nn.MSELoss()
```

Used for sentiment classification.

Question: Why wrong?

Answer:
MSE is for regression.

Fix:

```
loss = nn.CrossEntropyLoss()
```

Explanation:
Classification needs probability-based loss.

Q6. No Batching

Code:

```
for x, y in dataset:
   train(x, y)
```

Question: Problem?

Answer:
Training is slow.

Fix:

```
DataLoader(dataset, batch_size=32)
```

Explanation:
Batching speeds up training.

Q7. GPU Not Used

Code:

```
model = model
```

Question: Issue?

Answer:
Model runs on CPU.

Fix:

```
device = torch.device("cuda")
```

```
model = model.to(device)
```

Explanation:
GPU makes training faster.

Q8. FastAPI Deployment

Code:

```
@app.get("/predict")
def predict(text):
    return model(text)
```

Question: Problem?

Answer:
Raw text not processed.

Fix:

```
def predict(text):
    tokens = tokenize(text)
    return model(tokens)
```

Explanation:
Pipeline must be applied before prediction.

Q9. Model Not Saved

Code:

```
model.train()
```

Question: What missing?

Answer:
Model not saved.

Fix:

torch.save(model.state_dict(), "model.pth")

Explanation:
Needed for reuse/deployment.

Q10. LoRA Setup Issue

Code:

model = base_model

Question: Why inefficient?

Answer:
Full model training → high memory.

Fix:

model = get_peft_model(base_model, config)

Explanation:
LoRA trains small parameters only.

FINAL Revision of PART 2:

Topic	What it is	Key Idea	Example	Why Important

Text Pipeline	Steps to process text	Clean → Token → Convert to numbers	"I love AI" → [1,2,3]	Model cannot read raw text
Cleaning	Remove noise	Remove symbols, spaces	"Great!!!" → "Great"	Better data quality
Tokenization	Split text	Sentence → words	"I love AI" → ["I","love","AI"]	Basic input unit
Normalization	Standardize text	Lowercase, remove stopwords	"LOVE" → "love"	Reduce variation
Embeddings (Word2Vec/GloVe)	Words → vectors	Capture meaning	king ≈ queen	Semantic understanding
Padding	Equal length sequences	Add zeros	[1,2] → [1,2,0]	Needed for batching
Batching	Process multiple samples	Parallel training	batch size = 32	Faster training
RNN	Basic sequence model	Uses past info	"not good" issue	Simple but weak
LSTM	Advanced RNN	Uses memory gates	remembers "not"	Handles long context
GRU	Simplified LSTM	Fewer gates	faster than LSTM	Efficient model
BiLSTM	Two-directio n model	Past + future	better context	Better accuracy
Sentiment Analysis	Text classification	Positive/Negative	"Good movie"	Real-world use
Loss Function	Measure error	CrossEntropy	classification	Model learning
DataLoader	Load batches	Efficient data flow	batch training	Speed + memory
GPU Usage	Faster compute	CUDA	model.to(cuda)	Reduce training time

FastAPI	Deployment tool	API for model	/predict	Real-world usage
Inference Pipeline	Prediction flow	Text → model → output	user input	Production use
Caching	Store results	Reuse outputs	repeated query	Faster response
Async Queue	Handle requests	parallel API calls	many users	Scalability
MLOps	ML system design	Track + deploy	pipeline system	Industry use
Artifacts	Version control	Save dataset/model	v1, v2	Reproducibility
Monitoring	Track performance	accuracy, latency	production	Maintain quality
Rollback	Restore old model	if failure	revert version	Safety
PEFT	Efficient training	train small part	LoRA	Low memory
LoRA	Add small matrices	not full training	adapters	Fast + cheap
QLoRA	LoRA + compression	4-bit weights	NF4	Very low memory
Adapters	Add small layers	between layers	modular	Flexible
Quantization	Reduce size	16-bit → 4-bit	compression	Save memory
Experiment Tracking	Track runs	W&B	l	

PART 3

Transformers ,BERT, LLM & AutoML

FLOWCHART (TOPICS)

RNN Limitations → Self-Attention → Positional Encoding → Transformer Block → Mini-Transformer → BERT Architecture → BERT Fine-Tuning → Deployment

1. RNN LIMITATIONS (Why Transformers came)

RNN processes words one by one, so it is slow and forgets long context.

Example:
Sentence: "The movie which I watched yesterday was amazing"
RNN may forget "movie" when reaching "amazing".

---

2. SELF-ATTENTION

Each word looks at all other words and decides importance.

Example:
Sentence: "The cat sat on the mat"
Word "sat" focuses more on "cat" than "mat".

Simple idea:
Every word asks → Which words are important for me?

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SelfAttention(nn.Module):
    def __init__(self, d_model):
        super(SelfAttention, self).__init__()

        # Create Query, Key, Value matrices
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)

    def forward(self, x):
        # x shape: (batch_size, seq_len, d_model)
```

---

3. POSITIONAL ENCODING

Transformer does not know order of words, so we add position info.

Example:
"Dog bites man" ≠ "Man bites dog"
Without position, both look same → wrong meaning

Solution:
Add position numbers to each word vector.

---

4. TRANSFORMER BLOCK **(very simple + real sentence example)**

Example sentence: *"The cat sat on the mat"*

---

1. Attention layer
Meaning: Model focuses on important words in the sentence.
Example: For word **"sat"**, it looks at **"cat"** (who sat).
→ It connects *sat* ↔ *cat*

---

2. Feed Forward Network (FFN)
Meaning: Processes each word deeply (like thinking step).
Example: After attention, it understands **"sat" = action done by cat**
→ Adds meaning to each word separately

---

3. Residual connection (skip connection)
Meaning: Keeps original info + new info (don't forget basics)
Example: Word "sat" = old meaning + new context
→ Helps not lose original word meaning

---

4. Layer Normalization
Meaning: Keeps values stable (smooth learning)
Example: Makes sure no word becomes too strong or too weak
→ Balanced understanding of sentence

One-line flow:
Sentence → Attention (focus) → FFN (understand) → +Original (residual) → Normalize → Output

5. MINI-TRANSFORMER

Full model built using multiple transformer blocks.

Flow:
Input → Embedding → Position → Blocks → Output

Example:
Input sentence → model predicts positive/negative sentiment

Use when:

- Learning concepts
- Small dataset

6. BERT ARCHITECTURE

BERT = Big Transformer model (pre-trained)

Key idea:
Learn language first, then solve task

Features:

- Bidirectional (looks both left and right)
- Many layers (12 or 24)

tHE MOVIE WAS NOT GOOD= negative

Example:
Sentence: "He went to bank"
BERT uses full sentence to know "bank" meaning

MAIN ARCHITECTURE of BERT

FLOW (BERT)

Text → Tokenizer → Embedding → Transformer Encoder Layers → [CLS] → Classifier → Output

---

1. INPUT PROCESSING

BERT takes sentence and converts into tokens.

Example:
"I love AI" → `[CLS] I love AI [SEP]`

- `[CLS]` → used for classification
- `[SEP]` → end of sentence

---

2. EMBEDDINGS (3 types added)

Each token vector =
Token Embedding + Position Embedding + Segment Embedding

Example:

"I love AI"

- Token: meaning of word
- Position: word order
- Segment: sentence A or B

---

3. TRANSFORMER ENCODER (MAIN PART)

BERT = stack of encoder layers (12 or 24)

Each layer has:

1. Multi-head self-attention
2. Feed Forward Network

Working:

Each word looks at all other words.

Example:
Sentence: "He went to bank to deposit money"

Word "bank" attends to "deposit" → meaning = financial bank

---

4. SELF-ATTENTION (CORE IDEA)

Each word creates:

Query (Q), Key (K), Value (V)

Then:

Attention score = similarity(Q, K)

Important words get higher weight.

---

5. OUTPUT ([CLS] TOKEN)

Final output uses `[CLS]` vector.

Example:

Input: "Movie is very good"
[CLS] output → fed to classifier → Positive

---

6. PRE-TRAINING TASKS

1. MLM (Masked Language Model)

"I love [MASK]" → model predicts "AI"

2. NSP (Next Sentence Prediction)

Sentence A: "I am hungry"
Sentence B: "I ate food" → related (Yes)

---

7. WHY BERT IS POWERFUL

- Understands full context
- Uses both left + right words
- Pre-trained on huge data

SIMPLE REAL-LIFE ANALOGY

BERT = smart reader

- Reads whole sentence
- Understands meaning using context
- Then answers question

CODE EXAMPLE (BERT CLASSIFICATION)

Install

```
!pip install transformers torch
```

Load Model

```
from transformers import BertTokenizer, BertForSequenceClassification
import torch

# tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

```
# model (2 classes)
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2
)
```

Input Text

```
text = "This movie is amazing"

inputs = tokenizer(
    text,
    return_tensors='pt',
    padding=True,
    truncation=True
)
```

Prediction

```
outputs = model(**inputs)

logits = outputs.logits
prediction = torch.argmax(logits, dim=1)

print("Prediction:", prediction.item())
```

Output:

- 0 → Negative
- 1 → Positive

With Probability

probs = torch.softmax(logits, dim=1)
print(probs)

TRAINING (SIMPLE IDEA)

loss = outputs.loss
loss.backward()
optimizer.step()

FINAL SHORT SUMMARY

BERT =
Input → Embedding → Encoder Layers → [CLS] → Output

Uses self-attention to understand full sentence context.

PART 1: OBJECTIVE MCQs

Q1. What is the main role of the `[CLS]` token in BERT?

A. Marks sentence end
B. Stores sentence-level representation

C. Separates two sentences
D. Removes padding

Answer: B

Explanation:
`[CLS]` captures full sentence meaning and is used for classification tasks.

Q2. Why does BERT use positional encoding?

A. To reduce computation
B. To store word frequency
C. To preserve word order
D. To increase vocabulary

Answer: C

Explanation:
Transformer has no sequence order, so positional encoding tells the model the position of words.

Q3. In self-attention, what determines how much attention one word gives to another?

A. Word length
B. Query-Key similarity
C. Value vector size
D. Position index

Answer: B

Explanation:
Attention score is computed using similarity between Query and Key.

Q4. What makes BERT bidirectional?

A. Uses CNN
B. Uses RNN
C. Looks at both left and right context
D. Uses only forward pass

Answer: C

Explanation:
Unlike older models, BERT reads full sentence context at once.

Q5. What is the purpose of Masked Language Modeling (MLM)?

A. Translate text
B. Predict missing words
C. Classify sentences
D. Remove noise

Answer: B

Explanation:
Model learns language by predicting hidden words.

Q6. What happens if positional encoding is removed?

A. Model becomes faster
B. Model cannot understand word order
C. Model overfits
D. No effect

Answer: B

Explanation:
Without position, "dog bites man" = "man bites dog".

Q7. Why is learning rate kept very low in BERT fine-tuning?

A. To reduce memory
B. To avoid overfitting
C. To preserve pre-trained knowledge
D. To increase speed

Answer: C

Explanation:
High learning rate can destroy learned language features.

Q8. What is the output of BERT before classification?

A. Single number
B. Token embeddings
C. Image features
D. Audio signal

Answer: B

Explanation:
BERT outputs contextual embeddings for each token.

PART 2: CASE-BASED QUESTIONS

Case 1:

A model is given sentence:
"He went to the bank to deposit money"

Q9. How does BERT correctly understand "bank"?

A. Uses only previous word
B. Uses full sentence context
C. Uses dictionary lookup
D. Uses fixed embedding

Answer: B

Explanation:
BERT attends to "deposit" → understands financial meaning.

Case 2:

Sentence A: "I am tired"
Sentence B: "I will sleep"

Q10. What will NSP task predict?

A. Not related
B. Related
C. Random
D. Cannot predict

Answer: B

Explanation:
Both sentences logically connected → NSP = True.

Case 3:

Sentence: "The food was not good"

Q11. Why is BERT better than RNN here?

A. Faster training
B. Understands "not good" together
C. Uses fewer parameters
D. Ignores context

Answer: B

Explanation:
BERT captures dependency between "not" and "good".

Case 4:

You remove `[CLS]` token and try classification.

Q12. What happens?

A. Model improves
B. Model cannot classify properly

C. No change
D. Faster training

Answer: B

Explanation:
No sentence-level representation → classification fails.

Case 5:

You fine-tune BERT with very small dataset (100 samples).

Q13. Best strategy?

A. Train full model
B. Freeze BERT, train classifier only
C. Increase layers
D. Use high learning rate

Answer: B

Explanation:
Pre-trained knowledge is enough → avoid overfitting.

Case 6:

Two sentences:

A: "He is playing cricket"
B: "Bananas are yellow"

Q14. NSP output?

A. Related
B. Not related
C. Both
D. Unknown

Answer: B

Explanation:
No logical connection → NSP = False.

PART 3: CONCEPT SUMMARY

Key Concepts Tested

- Self-attention → relation between words
- Positional encoding → word order
- `[CLS]` → classification
- MLM → language understanding
- NSP → sentence relation
- Fine-tuning → task adaptation

FINAL ONE-LINE REVISION

BERT understands full sentence context using self-attention and is fine-tuned for tasks using `[CLS]` output.

Fine Tuning

HORIZONTAL FLOWCHART (TOPICS)

Fine-Tuning Challenge → Parameter Efficient Fine-Tuning (PEFT) → LoRA → Working of LoRA → QLoRA → Quantization → Optuna (Hyperparameter Tuning) → FAQ Fine-Tuning → Training Pipeline → Deployment & Best Practices

---

1. FINE-TUNING CHALLENGE

Large Language Models (LLMs) have billions of parameters.
Training or fine-tuning all parameters needs huge memory.

Example:
7B model needs ~100GB memory → not possible on normal GPUs.

Real life:
Like trying to repaint a whole building when you only need to fix one wall.

---

2. PEFT (PARAMETER EFFICIENT FINE-TUNING)

Idea:
Do not update full model → update only small part.

Why needed:

- Saves memory
- Faster training
- Works on small GPUs

Example:
Instead of changing full book, you add sticky notes on important pages.

---

3. LoRA (LOW-RANK ADAPTATION)

Key concept:
Weight update matrix is simplified into smaller matrices.

Instead of:
ΔW (big matrix)

We use:
$\Delta W = B \times A$

Where:

- A → reduces dimension
- B → expands back

Why useful:

- Very few parameters
- Same performance

Example:
Big matrix (1000×1000 = 1M params)
LoRA (1000×10 + 10×1000 = 20k params)

Real life:
Instead of replacing entire engine, add a small turbo part.

---

4. HOW LoRA WORKS IN MODEL

Final output:

Output = Original (frozen weights) + LoRA update

- Original model → not trained
- Only A and B → trained

Important:
Applied mainly on attention layers (Q, K, V, O)

Example:
Teacher knowledge stays same
Student adds extra notes to adapt to exam

---

5. QLoRA (QUANTIZED LoRA)

Problem with LoRA:
Still large model stored in memory.

Solution:
Use quantization (4-bit) to compress model.

Memory comparison:

- Normal → 28GB
- QLoRA → ~3.5GB

Example:
Compressing a large video into smaller size with minimal quality loss.

---

6. QUANTIZATION

Convert high precision numbers to low precision.

Types:

- FP32 (high memory)
- FP16
- INT8
- INT4 (used in QLoRA)

Important idea:
Weights are approximated, not exact.

Example:
Instead of storing 3.141592 → store 3.14

Real life:
Reducing image quality to save space but still recognizable.

---

7. OPTUNA (HYPERPARAMETER OPTIMIZATION)

Problem:
Too many combinations of parameters.

Example:
Learning rate, batch size, epochs, LoRA rank

Manual tuning = very slow

Solution:
Optuna automatically finds best values.

---

8. HOW OPTUNA WORKS

Steps:

1. Try some random values
2. Check performance
3. Learn which values are good
4. Try better combinations

Uses:
TPE (Tree-structured Parzen Estimator)

Example:
If learning rate 3e-5 works well → try near values like 2e-5 or 4e-5

Real life:
Like adjusting cooking recipe based on taste feedback.

---

9. FAQ FINE-TUNING

Goal:
Train model for specific domain (customer support, medical, etc.)

Data format:

Question → Answer

Example:

Q: How to reset password?
A: Click forgot password

Model learns pattern and answers new questions.

---

10. INSTRUCTION FORMAT

Convert data into structured format:

"### Question:
...

Answer:

..."

Why important:
Model learns how to respond properly.

Example:
Like training a student using proper question-answer format.

---

11. TRAINING PIPELINE

Step-by-step:

1. Load base model
2. Apply LoRA / QLoRA
3. Prepare dataset

4. Tokenize data
5. Train model
6. Evaluate performance

Example:
Like preparing, teaching, testing a student.

---

12. DEPLOYMENT

After training, model is used in real applications.

Examples:

- Chatbots
- Medical assistants
- FAQ systems

Optimization:

- Reduce size
- Increase speed

---

13. BEST PRACTICES

- Use LoRA rank (r = 8–16 for most tasks)
- Use proper learning rate (LoRA needs higher LR)
- Avoid overfitting (use dropout)
- Use good quality data

Example:
Better data → better answers

FINAL SIMPLE SUMMARY

Full fine-tuning = costly
PEFT = train small part
LoRA = add small matrices
QLoRA = compress model
Optuna = find best parameters
FAQ fine-tuning = task-specific learning
Deployment = real-world usage

5 CASE BASED to Revise Everything;

(**MASAI-style)**

CASE 1: SELF-ATTENTION (TRANSFORMER)

A model is given the sentence:
"The trophy does not fit in the suitcase because it is too big."

Q1. How does a Transformer correctly understand what "it" refers to?

A. Looks only at previous word
B. Uses positional encoding only
C. Uses self-attention to relate "it" with "trophy"
D. Uses fixed embeddings

Answer: C

Explanation:
Self-attention allows each word to look at all other words.
Here, "it" attends more to "trophy" because "too big" logically relates to trophy, not

suitcase.
This is why Transformers handle long dependencies better than RNNs.

CASE 2: POSITIONAL ENCODING

Two sentences are given:

1. "Dog bites man"
2. "Man bites dog"

Q2. What will happen if positional encoding is removed?

A. Model still understands difference
B. Model treats both sentences same
C. Model crashes
D. Model becomes faster but accurate

Answer: B

Explanation:
Without positional encoding, Transformer sees words as a set, not sequence.
So both sentences have same words → same meaning → wrong understanding.

CASE 3: BERT CONTEXT UNDERSTANDING

Sentence:
"He went to the bank to withdraw cash."

Q3. How does BERT understand the meaning of "bank"?

A. Uses only left context
B. Uses only right context
C. Uses both left and right context
D. Uses dictionary lookup

Answer: C

Explanation:
BERT is bidirectional.
It looks at "withdraw" and "cash" to understand that "bank" = financial institution.
This is the key advantage over older models.

CASE 4: BERT FINE-TUNING

You are given only 200 labeled samples for sentiment analysis.

Q4. What is the best strategy?

A. Train BERT from scratch
B. Fine-tune full BERT with high learning rate
C. Freeze most layers and train classifier
D. Use no pre-trained model

Answer: C

Explanation:
Small dataset → risk of overfitting.
Best approach:

- Use pre-trained BERT
- Freeze most layers
- Train only classifier

This preserves learned knowledge and avoids overfitting.

CASE 5: LLM + LoRA (REAL-WORLD SCENARIO)

A company wants to fine-tune a 7B model on a 24GB GPU.

Q5. Which method is most suitable?

A. Full fine-tuning
B. LoRA
C. QLoRA
D. Train from scratch

Answer: C

Explanation:
Full fine-tuning needs ~100GB memory → not possible.
LoRA reduces parameters but model still large.
QLoRA:

- Uses 4-bit quantization
- Reduces memory to ~9GB
- Works on 24GB GPU

Hence best practical solution.

FINAL CONCEPT SUMMARY

These questions test:

- Self-attention → relation understanding
- Positional encoding → word order
- BERT → bidirectional context
- Fine-tuning → data-efficient learning
- LoRA/QLoRA → memory-efficient training

ONE-LINE REVISION

Transformer understands relationships, BERT understands context, and LoRA/QLoRA makes large models trainable on small hardware.

AUTOML for LLMs

FLOWCHART (TOPICS)

Hyperparameter Problem → AutoML → Bayesian Optimization → Acquisition Function → Optuna → Early Stopping → Pruning → Search Space Design → Deployment Pipeline

---

HORIZONTAL FLOWCHART (TOPICS)

Hyperparameter Problem → AutoML → Bayesian Optimization → Acquisition Function → Optuna → Early Stopping → Pruning → Search Space Design → Deployment Pipeline

---

1. HYPERPARAMETER PROBLEM

In LLM training, many parameters need tuning:

- Learning rate
- Batch size
- Epochs
- Dropout

Problem:
Too many combinations → very slow and confusing.

Example:
Choosing best cooking recipe with many ingredients → difficult.

---

2. AUTOML

AutoML = automatic system to find best parameters.

Idea:
Model learns from previous experiments and improves.

Example:
Instead of guessing recipe, system learns which ingredients work best.

---

3. BAYESIAN OPTIMIZATION

Smart method to search best parameters.

Steps:

1. Try some values
2. Learn pattern
3. Predict better values
4. Try again

Example:
If learning rate 3e-5 works well → try nearby values.

Real life:
Like improving exam strategy after seeing previous results.

---

4. GAUSSIAN PROCESS (IDEA)

Model predicts performance for new parameters.

Output:

- Mean (expected result)
- Uncertainty (confidence)

Example:
"I think this setting gives 80% accuracy ±5%"

---

5. ACQUISITION FUNCTION

Decides next experiment.

Types:

- Expected Improvement (EI) → balanced
- UCB → more exploration

- PI → focus on best

Example:
Choosing next step based on best chance of improvement.

---

6. OPTUNA

Tool that implements AutoML.

What it does:

- Suggests parameters
- Runs training
- Keeps best result

Example:
Acts like smart assistant doing experiments automatically.

---

7. EARLY STOPPING

Stops training when model stops improving.

Why:

- Saves time
- Avoids overfitting

Example:

Epoch 1 → good
Epoch 2 → best
Epoch 3 → worse

Stop at epoch 2

Real life:
Stop studying when performance starts decreasing.

---

8. PRUNING (IN OPTUNA)

Stops bad experiments early.

Example:

Trial 1 → good
Trial 2 → very bad → stop early

Saves:

- Time
- Compute

Real life:
Stop a bad cooking attempt early instead of wasting ingredients.

---

9. SEARCH SPACE DESIGN

Choosing correct range of parameters.

Important rules:

- Learning rate → log scale
- Batch size → powers of 2
- Dropout → linear scale

Example:

Bad: random values
Good: informed range

Real life:
Search for house only in budget range, not everywhere.

---

10. TRAINING PIPELINE (AUTOML)

Steps:

1. Define search space
2. Run Optuna trials
3. Use early stopping
4. Prune bad trials
5. Select best parameters

Example:
Trial and improvement loop.

---

11. DEPLOYMENT

Use best model in real applications.

Example:

- Chatbots
- NLP systems

Optimize:

- Speed
- Accuracy

FINAL SIMPLE SUMMARY

Too many parameters → AutoML solves it →
Bayesian optimization learns best values →
Optuna automates process →
Early stopping saves time →
Pruning removes bad trials →
Search space improves efficiency

ONE-LINE REVISION

AutoML uses smart search (Bayesian optimization) to find best hyperparameters efficiently using Optuna, early stopping, and pruning.

Summary MCQs of Part 3

integrated MCQs (MASAI exam level) connecting **Transformer + BERT + LoRA/QLoRA + AutoML + Optuna**

Clean, conceptual, and case-based.

PART 1: CORE CONCEPT MCQs

Q1. Why were Transformers introduced over RNNs?

A. Reduce parameters
B. Process sequences in parallel and capture long dependencies
C. Use less memory
D. Work only for images

Answer: B

Explanation:
RNNs process sequentially and forget long context.
Transformers use self-attention → parallel + long-range dependency handling.

Q2. What is the main role of self-attention?

A. Reduce model size
B. Assign importance between words
C. Remove noise
D. Encode position

Answer: B

Explanation:
Each word attends to other words to understand relationships.

Q3. Without positional encoding, what happens?

A. Faster model
B. Same performance
C. Cannot distinguish word order
D. Model crashes

Answer: C

Explanation:
Transformer treats sentence as unordered set → wrong meaning.

Q4. What makes BERT different from basic Transformer?

A. Uses decoder
B. Bidirectional context understanding
C. Uses CNN
D. No attention

Answer: B

Explanation:
BERT reads both left and right context → better understanding.

Q5. What is the purpose of `[CLS]` token?

A. Separate sentences
B. Store sentence-level representation
C. Remove padding
D. Predict next word

Answer: B

Explanation:
Used for classification tasks.

PART 2: CASE-BASED (TRANSFORMER + BERT)

Q6. Sentence:

"The animal didn’t cross the road because it was too tired."

What helps model understand "it" refers to animal?

A. Positional encoding
B. Self-attention
C. Token embedding
D. Dropout

Answer: B

Explanation:
Self-attention links "it" with "animal" using context.

Q7. Sentence:

"Man bites dog" vs "Dog bites man"

Why BERT gives different meaning?

A. Uses CNN
B. Uses position + context
C. Uses only embeddings
D. Uses random weights

Answer: B

Explanation:
Position + bidirectional context preserves meaning.

PART 3: LoRA / QLoRA

Q8. What is the main idea of LoRA?

A. Train full model
B. Reduce dataset size
C. Train small low-rank matrices instead of full weights
D. Remove layers

Answer: C

Explanation:
$\Delta W = BA \rightarrow$ fewer parameters $\rightarrow$ efficient training.

Q9. Why is QLoRA better than LoRA in low-resource systems?

A. Faster training
B. Uses quantization (4-bit) to reduce memory
C. Uses more parameters
D. No training required

Answer: B

Explanation:
QLoRA compresses model → fits in small GPU.

Q10. You have 24GB GPU and want to fine-tune 7B model. Best method?

A. Full fine-tuning
B. LoRA
C. QLoRA
D. Train from scratch

Answer: C

Explanation:
QLoRA reduces memory (~9GB) → feasible.

PART 4: FINE-TUNING + STRATEGY

Q11. Small dataset (200 samples). Best approach?

A. Train from scratch
B. Full fine-tuning
C. Freeze model and train classifier
D. Increase epochs

Answer: C

Explanation:
Pre-trained knowledge used → avoids overfitting.

PART 5: AUTOML + OPTUNA

Q12. What problem does AutoML solve?

A. Data cleaning
B. Hyperparameter tuning
C. Model architecture design
D. Tokenization

Answer: B

Explanation:
AutoML finds best parameters automatically.

Q13. Why is Bayesian Optimization better than grid search?

A. Tries all combinations
B. Learns from previous results
C. Random selection
D. Faster hardware

Answer: B

Explanation:
Uses past results to guide next trials → efficient.

Q14. What is the role of Optuna?

A. Model training
B. Data preprocessing
C. Automated hyperparameter tuning
D. Visualization

Answer: C

Explanation:
Optuna runs trials and finds best parameters.

Q15. What is early stopping?

A. Stop training early randomly
B. Stop when validation performance stops improving
C. Stop after fixed epochs
D. Stop after loss becomes zero

Answer: B

Explanation:
Prevents overfitting and saves time.

Q16. What is pruning in Optuna?

A. Removing dataset
B. Stopping bad trials early
C. Reducing model layers
D. Reducing batch size

Answer: B

Explanation:
Bad experiments are stopped early → saves resources.

PART 6: SEARCH SPACE DESIGN

Q17. Why use log scale for learning rate?

A. Linear behavior
B. Multiplicative effect
C. Faster training
D. Less memory

Answer: B

Explanation:
Learning rate affects multiplicatively → log scale better.

Q18. Why use powers of 2 for batch size?

A. Easy to remember
B. GPU efficiency
C. Less memory
D. Faster convergence

Answer: B

Explanation:
Hardware optimized for powers of 2.

FINAL INTEGRATED CASE

Q19. You are building a chatbot with limited GPU. You choose:

- BERT model
- LoRA

- Optuna
- Early stopping

What is correct pipeline?

A. Train full model → tune manually
B. Use LoRA → AutoML (Optuna) → early stopping
C. Use CNN → random search
D. Skip tuning

Answer: B

Explanation:
Correct modern pipeline:

- LoRA → efficient training
- Optuna → best parameters
- Early stopping → efficient training

FINAL ONE-LINE REVISION

Transformers learn relationships, BERT understands context, LoRA/QLoRA enable efficient training, and AutoML (Optuna) finds best parameters automatically.

FINAL SUMMARY QUESTIONS.

Final MIXED MCQs (Exam Level)

Q1. Why CNN is used for images and not ANN?

A. Faster training
B. Keeps spatial information
C. Uses less data
D. No layers

Q2. In CNN, which layer reduces size but keeps important info?

A. Convolution
B. Pooling
C. FC
D. Padding

Q3. In Transformer, which part finds relation between words?

A. FFN
B. Attention
C. Normalization
D. Embedding

Q4. Sentence: "The cat sat because it was tired"
"it" refers using:

A. CNN
B. Self-attention
C. Pooling
D. Padding

Q5. Why positional encoding is important?

A. Reduce loss

B. Preserve word order
C. Remove tokens
D. Increase layers

Q6. In BERT, meaning of “bank” is understood using:
A. Only left word
B. Only right word
C. Full context
D. Dictionary

Q7. Which model handles long sentence dependency best?
A. CNN
B. RNN
C. Transformer
D. ANN

Q8. What problem does pruning solve?
A. Overfitting
B. Large model size
C. Tokenization
D. Padding

Q9. Layer normalization mainly helps in:
A. Speed only
B. Stable learning
C. Reducing data
D. Removing layers

Q10. YOLO vs SSD main difference:

A. CNN vs RNN
B. Grid vs multi-scale detection
C. Speed vs memory
D. Pooling vs FC

Q11. In NLP pipeline, tokenization does:

A. Remove noise
B. Split text into words
C. Train model
D. Normalize data